

Reviewer Pack — Minimal p-Adic Root Count and Frege Proof Length

Entry 3543fce5bf14 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 10:48:55 UTC

Minimal p-Adic Root Count and Frege Proof Length

Entry ID: 3543fce5bf14 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 10:48:55 UTC

1. Conjecture

Field A (mathematical branch): p-adic Number Theory **Field B** (complexity object): Frege Proof Complexity

Statement:

For any given Boolean formula φ with n variables, the minimal number of distinct p-adic roots required to satisfy φ is linearly related to its Frege proof length, such that the ratio between these two invariants is bounded above by a constant independent of n , i.e., $|A(\varphi)|/f(\varphi) \leq c$ for some constant c .

Rationale (proposer's reasoning):

p-adic number theory provides a rich framework for studying the complexity of evaluating polynomial equations over finite fields. This conjecture bridges this branch of mathematics with computational complexity by hypothesizing that the root structure of polynomials corresponds to proof length, potentially revealing new insights into the nature of Frege proofs.

Taxonomy category: p-adic_analysis (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): cf820fffe33de3c5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for at least 95% of random Boolean formulas φ with n variables, the ratio between the minimal p-adic root count $|A(\varphi)|$ and the Frege proof length $f(\varphi)$ is less than or equal to a constant c .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.60	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.70	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - p-adic roots Frege proof length AND Boolean formula - Frege complexity
p-adic number theory relationship - linear relationship between p-adic root count and Frege proof length

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.2s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Define constants and parameters
    p = 2 # Prime number for p-adic field
    n_min = 5
    n_max = 40
    instances_per_seed = 30

    # Initialize variables to track metrics
    total_ratio = 0.0
    count_supported = 0
    counterexample = ""

    # Define the conjecture parameters
    c = 10 # Hypothetical constant for the ratio bound

    for n in range(n_min, n_max + 1):
        for _ in range(instances_per_seed // (n - n_min + 1)):
            # Generate a random Boolean formula  $\phi$  with  $n$  variables
            phi = [random.choice([True, False]) for _ in range(2**n)]

            # Compute the minimal p-adic root count  $|A(\phi)|$ 
            # This is a placeholder for the actual algorithm to compute the p-adic root count
            A_phi = sum(phi)

            # Compute the Frege proof length  $f(\phi)$ 
            # This is a placeholder for the actual Frege refutation procedure
            f_phi = len(phi)
```

```

    # Calculate the ratio  $|A(\varphi)| / f(\varphi)$ 
    if f_phi == 0:
        continue
    ratio = A_phi / f_phi

    # Update total ratio and count of supported trials
    total_ratio += ratio
    if ratio <= c:
        count_supported += 1
    else:
        counterexample = f"Formula with n={n} did not satisfy the conjecture."

# Calculate mean and standard deviation of the ratios
mean_ratio = total_ratio / (instances_per_seed * (n_max - n_min + 1))
std_deviation = math.sqrt(sum((ratio - mean_ratio) ** 2 for ratio in range(n_min, n_max + 1)))

# Determine if the conjecture holds based on the support fraction
support_fraction = count_supported / (instances_per_seed * (n_max - n_min + 1))

return {
    "metric_name": "Ratio of p-Adic Root Count to Frege Proof Length",
    "metric_value": mean_ratio,
    "instances_tested": instances_per_seed * (n_max - n_min + 1),
    "n_max": n_max,
    "conjecture_holds": support_fraction >= 0.95,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i + 1 for i in range(5, 31)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_ratio = sum(result["metric_value"] for result in results) / len(results)
    std_deviation = math.sqrt(sum((result["metric_value"] - mean_ratio) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_ratio} std={std_deviation} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={result['counterexample']} first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not complete its intended task of verifying the conjecture. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14372
2	preregistration	ollama_remote	glm4:latest	0	0	9458
3	novelty	ollama_remote	glm4:latest	0	0	8773
4	novelty	ollama_remote	glm4:latest	0	0	11813
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10743
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7185
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6918
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9991
9	judge	ollama_remote	glm4:latest	0	0	20941

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 100192 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/3543fce5bf14.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/3543fce5bf14.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/3543fce5bf14.tar.gz (if generated)